We normalize the document content to improve summarization accuracy by ensuring the summarization algorithm focuses on the core substance of the documents and to reduce computational processing costs by performing upfront content-cleaning steps.

Here are three remedies to try:

- *Removing headers and footers*—During or after parsing of the documents, you can preprocess the results to remove extraneous content such as headers, footers, page numbers, and watermarks that do not contribute to the main text. Bear in mind, though, that this process is harder to do for PDFs compared to DOCX documents since headers and footers are simply text elements in a page.
- *Removing stopwords*—Another important aspect of cleaning up documents is the removal of stopwords—common words that do not carry significant meaning (like "and," "the," "is"). By filtering out these words, you can reduce noise in the data and reduce the number of tokens that you use for summarization.
- *Extracting images*—The current implementation of the `PDFLoader` does not recognize or extract images from PDF files, resulting in these visual elements being overlooked during the parsing process. To enhance the functionality of the loader, an alternative solution involves extracting images from the PDF and marking their locations within the document. Once the images are extracted, you can utilize optical character recognition software or an external LLM to generate descriptive text for each image. This text can then be attached to the corresponding document instance as metadata.

These solutions aim to address the current limitations that you may face when processing documents for summarization and analysis. I've also included a helper function, `normalizeDocuments`, in the summarization application folder (ch07/summarization-examples). This function streamlines the normalization of document content by implementing various cleaning techniques, and you are free to extend it as you see fit. The following listing shows its main functions.

Listing 7.3 Normalization of document content

```
function normalizeDocuments(docs) {
  return docs.map((doc) => {
    let pageContent;
    if (typeof doc.pageContent === "string") {
      pageContent = doc.pageContent;
    } else if (Array.isArray(doc.pageContent)) {
      pageContent = doc.pageContent.join("\n");
    }
    if (pageContent) {

      pageContent = pageContent.trim();

      // Remove unwanted characters (e.g., figure references)
      pageContent = pageContent.replace(/Figure \d+:.*?\n/g, '').replace(/-\
➥d+\n/g, '');
```

Consolidates the page content if it's stored as an array of strings, by joining the array elements with newline characters

Removes any leading or trailing whitespace from the page content